

RAPPORT D'ETAT DU PROJET

ARPT Guinee

Plateforme de regulation des telecommunications de la Republique de Guinee. Bilan complet de l'audit qualite, corrections realisees et feuille de route pour la mise en production.

Mai 2026

Phase 1 & 2 terminees

56 corrections sur 134 anomalies

Table des matieres

1. Presentation du Projet ARPT	2
1.1 Fiche technique du projet	2
2. Resultats de l'Audit Qualite	3
2.1 Repartition des anomalies par criticite	3
3. Phase 1 - Corrections Critiques (Realisee)	4
3.1 Validation des entrees et prevention de l'assignement en masse .	4
3.2 Durcissement des secrets de chiffrement	4
3.3 Correction du controle RBAC	5
3.4 Limitation du debit (Rate Limiting)	5
3.5 Correction du tableau de bord	5
4. Phase 2 - Corrections Haute Priorite (Realisee)	6
4.1 Assainissement des entrees et protection XSS	6
4.2 Securite des sessions et authentification	6
4.3 Gestion centralisee des erreurs	7
4.4 Pagination, CORS et securite des cookies	7
4.5 Stabilite de l'interface utilisateur	7
5. Phase 3 - Corrections Moyenne Priorite (En cours)	8
6. Phase 4 - Ameliorations Continues (A venir)	9
7. Metriques Cles et Avancement Global	10
8. Prochaines Etapes et Feuille de Route	11

1. Presentation du Projet ARPT

L'Autorite de Regulation des Postes et Telecommunications (ARPT) de la Republique de Guinee a initie le developpement d'une plateforme numerique integree destinee a superviser et reguler l'ensemble des activites du secteur des telecommunications du pays. Cette plateforme constitue un outil strategique pour le suivi de la qualite de service (QoS), la gestion des operateurs, le traitement des reclamations des abones, et la production de rapports reglementaires.

Le projet ARPT repose sur une architecture moderne combinant un frontend Next.js et un backend Express.js, tous deux interfaces avec une base de donnees PostgreSQL via l'ORM Prisma. La plateforme implemente un systeme d'authentification double (NextAuth + JWT Express), un controle d'accès basé sur les rôles (RBAC) avec quatre profils distincts (Directeur General, Chef de Service, Agent, Operateur), et un ensemble de 28 groupes de routes couvrant l'ensemble des domaines fonctionnels de la regulation.

Le perimetre fonctionnel couvert par l'application inclut la gestion des operateurs de telecommunications, le suivi des dossiers et decisions reglementaires, la gestion des declarations et reclamations, la generation de rapports et modeles de rapports, le suivi de la couverture reseau et des zones blanches, la configuration des seuils de qualite de service, la gestion des sanctions, la planification des audits, ainsi que la gestion des parametres systeme et des donnees ouvertes. Chacun de ces modules est expose via des API RESTful securisees avec validation Zod et autorisation RBAC.

1.1 Fiche technique du projet

Composant	Description
Frontend	Next.js 14+ (React), TypeScript, Tailwind CSS
Backend	Express.js, TypeScript, Prisma ORM
Base de donnees	PostgreSQL (production), SQLite (developpement)
Authentification	NextAuth (cookie session) + Express JWT (cookie arpt-session)
Autorisation	RBAC matriciel : 4 roles x 14 ressources x 4 actions
Validation	Zod schemas (18+ schemas couvrant toutes les routes POST/PUT)
Conteneurisation	Docker Compose (frontend + backend + PostgreSQL)
Depot source	GitHub : github.com/skaba89/arpt_app

Modeles Prisma	33 modeles (backend), 32+ modeles (frontend)
Routes API	28+ groupes de routes RESTful

Tableau 1 - Fiche technique synthetique du projet ARPT

2. Resultats de l'Audit Qualite

Un audit qualite exhaustif de bout en bout a ete realise sur l'ensemble de la plateforme ARPT. Cet audit a couvert les dimensions de securite, de robustesse fonctionnelle, de performance, d'accessibilite et de qualite du code. Le resultat global est sans appel : la plateforme a reçu le verdict de **NON VALIDEE POUR LA PRODUCTION**, avec un total de 134 anomalies identifiees reparties en quatre niveaux de criticite.

La repartition des anomalies par niveau de severite revele une proportion preoccupante de defects critiques et majeurs, representant ensemble 42% du total des anomalies constatees. Les problemes de securite dominent les anomalies critiques (absence de validation des entrees, secrets codes en dur, controle RBAC defaillant), tandis que les anomalies de priorite elevee concernent principalement des risques d'injection XSS, des crashes a l'execution, et l'absence de pagination sur les endpoints non bornes.

2.1 Repartition des anomalies par criticite

Severite	Nombre	Proportion	Exemples typiques
Critique	20	15%	Failles de securite majeures, validation absente, secrets exposes
Haute	36	27%	XSS, crashes runtime, pagination absente, CORS permissif
Moyenne	44	33%	Qualite code, accessibilite, etats de chargement manquants
Basse	34	25%	Optimisations, refactorisation, ameliorations mineures

Tableau 2 - Repartition des 134 anomalies identifiees par l'audit QA

L'audit a egalement souligne plusieurs problemes transversaux affectant l'ensemble de l'application : l'absence de gestion centralisee des erreurs, des secrets de chiffrement hardcodes dans le code source, des schemas de validation

Zod incomplets ou absents sur de nombreuses routes, une matrice RBAC incohérente entre le frontend et le backend, et l'absence de protection contre les attaques par force brute sur les endpoints d'authentification. Ces constats ont motivé l'adoption d'une approche de correction progressive en quatre phases distinctes.

3. Phase 1 - Corrections Critiques (Realisee)

La première phase de correction a ciblé les 20 anomalies de sévérité critique identifiées par l'audit. Ces anomalies représentaient des risques directs pour la confidentialité, l'intégrité et la disponibilité des données de la plateforme. La correction de ces failles constitue un prérequis indispensable avant toute mise en production. Deux sessions de travail ont été nécessaires pour couvrir l'ensemble des points critiques, avec un total de 39 fichiers modifiés, 521 insertions et 190 suppressions de lignes.

3.1 Validation des entrées et prévention de l'assignement en masse

L'audit a révélé que 20 routes POST et PUT du backend n'appliquaient aucune validation sur les données entrantes, permettant potentiellement l'assignement en masse (mass assignment). Des attaquants pourraient exploiter cette faille pour modifier des champs sensibles comme le rôle d'un utilisateur ou le statut d'une décision sans autorisation. La correction a consisté à créer 18 schémas de validation Zod couvrant l'ensemble des routes vulnérables, incluant des validateurs stricts pour les types, longueurs, et formats de données attendus. Les schémas `z.any()` ont été remplacés par des validateurs précis (`z.string().max()` pour les champs JSON, par exemple).

3.2 Durcissement des secrets de chiffrement

Les secrets JWT et NEXTAUTH étaient codés en dur dans le code source avec des valeurs faibles et prédictibles ('arpt-guinee-fallback-secret-for-dev-only', 'arpt-guinea-dev-secret-change-in-production-2025'). La correction a supprimé tous les secrets de repli (fallback secrets), généré des clés cryptographiques fortes (64 octets en base64), et migré toutes les références vers des variables d'environnement (JWT_SECRET, NEXTAUTH_SECRET, SESSION_SECRET). Le fichier docker-compose.yml a été mis à jour pour injecter ces secrets de manière

securisee dans les conteneurs. De plus, le flag de securite des cookies est desormais configurable via NEXTAUTH_SECURE_COOKIES pour s'adapter aux environnements de production avec HTTPS.

3.3 Correction du controle RBAC

Plusieurs routes exposaient des operations sensibles sans verification d'autorisation adequat. Cinq routes de rapports-generes utilisaient erroneement `authorize('templates')` au lieu de `authorize('rapports-generes')`. La route `POST /restore` des versions utilisait `authorize('versions','read')` au lieu de `authorize('versions','update')`. La route des antennes n'appliquait que `requireAuth()` sans verification RBAC. La matrice RBAC a ete corrige pour ajouter la permission `'update'` sur la ressource `'versions'` pour le role `'agent'`, necessaire pour l'operation de restauration.

3.4 Limitation du debit (Rate Limiting)

L'audit a identifie l'absence totale de protection contre les attaques par force brute et les abus de l'API. Un middleware de limitation du debit a ete cree avec trois niveaux de protection distincts : un limiteur global pour l'ensemble des endpoints API (300 requetes par minute), un limiteur specifique aux endpoints d'authentification (20 requetes par minute), et un limiteur pour les exports de donnees (30 requetes par tranche de 5 minutes). Ces limites protegent efficacement contre les attaques par deni de service, les tentatives de brute force sur les mots de passe, et l'extraction massive de donnees.

3.5 Correction du tableau de bord

Le tableau de bord principal affichait une valeur hardcodee pour le delai moyen de traitement des dossiers (`deloiMoyen = 4.2`), sans aucun lien avec les donnees reelles. Cette valeur a ete remplacee par un calcul dynamique base sur les dossiers existants en base de donnees, garantissant ainsi l'exactitude des indicateurs presentes aux decideurs.

Domaine	Probleme identifie	Correction appliquee
Validation entrees	20 routes sans validation	18 schemas Zod, toutes routes couvertes

Secrets de chiffrement	Secrets hardcodes, fallbacks insecure	Env vars, cles 64-byte, plus de fallback
Controle RBAC	5 routes avec autorisation incorrecte	Matrice RBAC corrige, permissions alignees
Rate Limiting	Aucune protection	3 niveaux : API 300/min, Auth 20/min, Export 30/5min
Tableau de bord	deloiMoyen = 4.2 (hardcode)	Calcul dynamique depuis la base de donnees

Tableau 3 - Synthese des corrections critiques (Phase 1)

4. Phase 2 - Corrections Haute Priorite (Realisee)

La deuxieme phase a adresse les 36 anomalies de priorite elevee, couvrant des problemes de securite complementaires, de stabilite du runtime, et de robustesse architecturale. Ces corrections ont ete reparties en deux vagues successives, representant au total plus de 1 500 lignes modifiees a travers 34 fichiers. Les corrections apportees dans cette phase renforcent significativement la posture de securite de la plateforme et elimine les principaux vecteurs d'attaque identifies.

4.1 Assainissement des entrees et protection XSS

Un middleware d'assainissement automatique a ete implemente pour strip les balises HTML, les gestionnaires d'evenements JavaScript (onclick, onerror, etc.), les URIs javascript: et data: de toutes les entrees utilisateur sur les requetes POST, PUT, PATCH et DELETE. Cette protection s'applique globalement apres le parsing du body, garantissant qu'aucune donnee malveillante ne puisse atteindre les handlers de routes. En complement, une vulnerabilite XSS dans le composant chatbot a ete corrige par l'integration de DOMPurify pour assainir le contenu HTML avant son rendu dans le navigateur.

4.2 Securite des sessions et authentication

Plusieurs ameliorations majeures ont ete apportees au systeme d'authentification. Le type du champ User.role a ete converti de String a enum Prisma (Role { dg, agent, operateur, public }), garantissant au niveau de la base de donnees que seuls les roles valides peuvent etre attribues. Une attaque temporelle sur la

verification des sessions a ete eliminee en utilisant `crypto.timingSafeEqual` pour la comparaison des tokens, empechant toute attaque par canal auxiliaire. Les tokens JWT en liste noire sont desormais persistes en base de donnees (modele `BlockedToken`) plutot qu'en memoire, assurant leur invalidation meme apres un redemarrage du serveur.

Un mecanisme de changement obligatoire du mot de passe a ete ajoute via le champ `doitChangerMotDePasse` sur le modele `User` (default: `true`). Lors de la connexion, si ce champ est actif, l'utilisateur est redirige vers l'ecran de changement de mot de passe avant tout acces a l'application. Ce mecanisme garantit que tous les comptes par default sont forces a personnaliser leur mot de passe lors de la premiere connexion.

4.3 Gestion centralisee des erreurs

L'audit a constate l'absence de gestion uniforme des erreurs a travers les 28 groupes de routes. Chaque handler de route implementait sa propre logique `try/catch` avec des formats de reponse incoherents. Un utilitaire `asyncHandler` a ete cree pour encapsuler automatiquement les handlers asynchrones, capturant les erreurs non gereses et retournant des reponses au format standardise. Cela garantit que toutes les erreurs de l'API retournent un format JSON coherent incluant le code d'erreur, le message, et le timestamp de l'incident.

4.4 Pagination, CORS et securite des cookies

L'ensemble des endpoints retournant des listes non bornees a ete dote de parametres de pagination (`limit/offset`) pour prevenir les denis de service par requetes massives et optimiser les performances de l'API. La configuration CORS a ete renforcee avec une liste blanche stricte des origines autorisees, rejetant les origines inconnues avec un log d'avertissement. Le debug endpoint `/api/debug/` a ete retire des exemptions de rate limiting, et les headers de securite HSTS ont ete ajoutes pour forcer les connexions HTTPS en production. Les cookies de session beneficient desormais des attributs `Secure`, `HttpOnly` et `SameSite=Strict` de maniere systematique.

4.5 Stabilté de l'interface utilisateur

Trois crashes runtime causes par des references a des proprietes indefinies (`pag undefined`) ont ete corriges dans les composants de pagination. Le composant `PaginationBar` a ete reecrit avec `React.memo`, un `displayName` explicite, et des

exports doubles pour une meilleure compatibilite. Un composant `SectionErrorBoundary` a ete ajoute, enveloppant les 27 cas de vue dans le composant principal de l'application, garantissant qu'une erreur dans une section n'entraîne pas l'écran blanc de l'ensemble de l'interface. Des regles CSS responsives pour mobile ont ete ajoutees (grille empilable, prevention des debordements), et la mise en page de la page de connexion a ete corrigeée pour les ecrans desktop.

Domaine	Probleme identifie	Correction appliquee
Assainissement XSS	Injection HTML/JS possible	Middleware sanitize + DOMPurify
Enum Role Prisma	User.role = String (non constraint)	Enum Role { dg, agent, operateur, public }
Attaque temporelle	Comparaison tokens non constante	crypto.timingSafeEqual
JWT blocklist	Tokens en memoire volatile	Modele BlockedToken en BDD
Changement mot de passe	Pas de rotation initiale	doitChangerMotDePasse (default: true)
Gestion erreurs	try/catch incoherents	asyncHandler centralise
Pagination API	Endpoints non bornes	limit/offset sur tous les endpoints
CORS	Origines permissives	Whitelist stricte + log rejets
HSTS & cookies	Headers manquants	HSTS + Secure + HttpOnly + SameSite
UI stability	3 crashes runtime, pas de fallback	React.memo + ErrorBoundary + CSS mobile
Indexes BDD	18+ indexes manquants	Indexes sur User.role, Declaration, etc.
Fichiers morts	proxy.ts inutilise	Supprime

Tableau 4 - Synthese des corrections haute priorite (Phase 2)

5. Phase 3 - Corrections Moyenne Priorite (En cours)

La phase 3 cible les 44 anomalies de priorite moyenne, representant le tiers des defects identifies. Ces anomalies n'affectent pas directement la securite ou la

stabilite de l'application, mais impactent significativement la qualite du code, l'experience utilisateur, et la maintenabilite a long terme de la plateforme. Les corrections de cette phase sont estimees a environ 20 heures de travail et constituent une etape essentielle avant la mise en production.

Les categories principales d'anomalies moyenne priorite incluent l'amelioration de la qualite du code TypeScript (typage incomplet, any implicites), l'ajout d'etats de chargement et de messages d'erreur contextuels dans les composants frontend, l'amelioration de l'accessibilite (contraste, navigation clavier, labels ARIA), la reduction de la duplication de code entre les composants, l'optimisation des requetes base de donnees (N+1 queries), et l'ajout de tests unitaires pour les fonctions critiques du backend.

Categorie	No mbr e	Description des anomalies
Qualite du code	12	Typage TypeScript incomplet, any implicites, duplication
Etats de chargement	8	Absence de spinners/skeletons pendant les appels API
Accessibilite	7	Contraste insuffisant, labels ARIA manquants, navigation clavier
Performance	6	N+1 queries, re-rendus inutiles React, bundles non optimises
Messages d'erreur	5	Messages generiques, pas de guidance utilisateur
Tests unitaires	4	Couverture de tests inferieure a 10%
Documentation API	2	Endpoints non documentes, schemas OpenAPI absents

Tableau 5 - Repartition des anomalies de priorite moyenne (Phase 3)

6. Phase 4 - Ameliorations Continues (A venir)

La phase 4 regroupe les 34 anomalies de priorite basse, representant les ameliorations souhaitables mais non bloquantes pour la mise en production. Ces items incluent des optimisations de performance avancees, de la refactorisation de code pour une meilleure maintenabilite, l'ajout de fonctionnalites cosmetiques, et des ameliorations de l'experience developpeur. L'estimation de travail pour

cette phase est d'environ 10 heures. Bien que ces ameliorations puissent etre differees apres le lancement, elles contribuent a la qualite a long terme et a la dette technique du projet.

Parmi les ameliorations prevues, on compte la mise en cache des requetes frequentes (Redis), l'optimisation des images et assets statiques, la refactorisation des composants React dupliques en composants partages, l'ajout de logs structures pour le monitoring en production, l'implementation d'un systeme de notifications en temps reel (WebSocket), et la completion de la documentation technique et fonctionnelle de l'application.

7. Metriques Cles et Avancement Global

Le suivi de l'avancement des corrections est essentiel pour evaluer la proximite de la plateforme avec les exigences de mise en production. Les metriques ci-dessous presentent l'etat actuel du projet en termes de corrections realisees, d'effort investi, et de travail restant. Sur les 134 anomalies initiales, 56 ont ete corrigees a l'issue des phases 1 et 2, representant 42% du total et la quasi-totalite des anomalies critiques et haute priorite.

Indicateur	Valeur	Remarque
Anomalies totales identifiees	134	-
Anomalies corrigees (Phase 1 + 2)	56	42%
Anomalies critiques corrigees	20 / 20	100%
Anomalies haute priorite corrigees	36 / 36	100%
Anomalies moyenne priorite restantes	44	Phase 3 en cours
Anomalies basse priorite restantes	34	Phase 4 a venir
Fichiers modifies (Phase 1)	39	521 insertions, 190 suppressions
Fichiers modifies (Phase 2)	34	1 500+ lignes modifiees
Schemas Zod ajoutes	18	Couverture complete des routes
Indexes base de donnees ajoutes	18+	User.role, Declaration, Notification, etc.

Commits pousses sur GitHub	4	4215d5b, 1346ea8, b9333ff, 5365665
----------------------------	---	------------------------------------

Tableau 6 - Metriques cles du projet ARPT

8. Prochaines Etapes et Feuille de Route

La feuille de route du projet ARPT definit les prochaines etapes necessaires pour atteindre le niveau de qualite requis pour la mise en production. L'objectif prioritaire est l'achevement de la Phase 3 (44 anomalies moyenne priorite), qui permettra de lever la majorite des obstacles restants a la mise en service. La Phase 4 pourra etre menee en parallele avec les premiers deploiements en environnement de pre-production.

E t a p e	Action	Description	Effort	Dependance
1	Achevement Phase 3	Correction des 44 anomalies moyenne priorite	~20h	Priorite maximale
2	Tests de regression	Validation que les corrections n'introduisent pas de regressions	~8h	Apres Phase 3
3	Audit de securite post-correction	Verification que toutes les failles sont colmatees	~6h	Apres tests
4	Phase 4 (ameliorations)	Correction des 34 anomalies basse priorite	~10h	En parallele
5	Deploiement pre-production	Mise en service en environnement de staging	~4h	Apres audit
6	Formation utilisateurs	Formation des agents ARPT a la plateforme	~8h	Apres deploiement
7	Mise en production	Deploiement en environnement de production	~4h	Decision DG

Tableau 7 - Feuille de route du projet ARPT

L'effort total restant est estime a environ 60 heures de travail, reparties entre les corrections de la Phase 3 (20h), les tests de regression (8h), l'audit post-correction (6h), la Phase 4 (10h), le deploiement (4h), et la formation (8h). En

mobilisant les ressources adéquates, la mise en production pourrait être envisagée dans un délai de 3 à 4 semaines, sous réserve de la validation de l'audit de sécurité post-correction et de l'approbation formelle du Directeur Général de l'ARPT.